

기술 프레임워크 요약

이 프로젝트는 프론트엔드와 백엔드가 분리된 웹 애플리케이션입니다. 프론트엔드는 화면과 사용자 상호작용을, 백엔드는 대화 로직·안전 검사·데이터 저장·AI 모델 호출을 담당합니다. 둘은 HTTP API로 통신합니다.



1. 프론트엔드

구분	사용 기술	용도
프레임워크	Vue 3 (Composition API, <code><script setup></code>)	페이지와 컴포넌트
언어	TypeScript	타입 검사로 단순 실수 방지
빌드	Vite	로컬 개발 서버와 번들링
라우팅	Vue Router	페이지 전환, 페이지별 지연 로딩
상태 관리	Pinia	로그인 상태(<code>stores/auth.ts</code>), 현재 대화(<code>stores/session.ts</code>)
다국어	vue-i18n	중국어 / 한국어 문구, <code>src/i18n/index.ts</code> 에서 일괄 관리
네트워크 요청	axios	<code>src/api/client.ts</code> 에 통합
코드 검사	vue-tsc, ESLint, oxlint	타입 검사와 코드 규칙
배포	Cloudflare Workers (정적 자산)	<code>npm run deploy</code> , 설정은 <code>wrangler.jsonc</code>

디렉터리 구조

frontend/src/	
├─ api/client.ts	모든 백엔드 API 호출과 데이터 타입
├─ views/	페이지: 채팅, 대화 기록, 내 기록, 도움말, 로그인·회원가입, 관리자 페이지
├─ components/	공통 컴포넌트: 메뉴, 사이드바, 사전 안내(동의) 대화상자, 선택 카드 등
├─ admin/	관리자 페이지의 「AI 대화 설정」 편집기
└─ stores/	전역 상태 (로그인, 현재 대화)

i18n/	중·한 문구와 언어 설정
router/	라우팅과 관리자 페이지 접근 제한
assets/main.css	전역 스타일 변수 (색상, 모서리, 그림자)

페이지 목록

경로	페이지	로그인 필요 여부
/	채팅 (시작 시 사전 안내 포함)	예
/history	대화 기록	예
/records	내 기록 (해당 사용자의 모든 대화에서 저장한 기록, 유형별 필터 가능)	예
/reflection	매일 사용 성찰일지	예
/help	도움말	아니요
/privacy	개인정보 처리 안내	아니요
/contact	문의하기	아니요
/login /register	로그인, 회원가입 (아이디 + 비밀번호)	아니요
/admin	관리자 페이지	예, 관리자만 접근 가능

익명 사용은 없습니다. 로그인하지 않은 상태로 로그인이 필요한 페이지에 접근하면 로그인 페이지로 이동하며, 로그인 후 원래 페이지로 돌아갑니다. 백엔드 API도 로그인을 요구하며, 사용자는 자신의 대화·기록·성찰일지에만 접근할 수 있습니다.

2. 백엔드

구분	사용 기술	용도
언어	Python 3.12	
프레임워크	FastAPI + Uvicorn	API 제공, 비동기 요청 처리
데이터 검증	Pydantic / pydantic-settings	요청·응답 형식 검증, .env 설정 읽기
데이터베이스	SQLite (aiosqlite) + SQLAlchemy 2 (비동기)	데이터 파일 위치: <code>backend/data/app.db</code>
AI 모델	OpenAI API (기본값 <code>gpt-4o-mini</code>)	매 턴의 답변과 선택지 생성
로그인 인증	아이디 + 비밀번호, JWT (PyJWT) + 비밀번호 해시	이메일 미사용. 로그인 유지 30일, 사용 중지된 계정은 즉시 만료

구분	사용 기술	용도
데이터 암호화	cryptography (Fernet)	사용자가 저장한 기록을 DB에 암호화하여 보관
의존성 관리	uv	pyproject.toml + uv.lock
배포	Docker (Dockerfile, docker-compose.yml)	데이터 디렉터리를 볼륨으로 연결해 재시작해도 데이터 유지

디렉터리 구조

backend/app/	
├─ main.py	앱 진입점, 모든 API 등록
├─ api/	API 계층 (기능별 파일)
├─ services/	비즈니스 로직
│ └─ dialogue_state.py	대화 흐름 총괄: 안전 검사 → 흐름 선택 → 모델 호출 → 결과 정리
├─ safety.py	위기 키워드 검사 (AI가 아닌 코드로 판단)
├─ llm.py	OpenAI 호출, 모델이 항상 JSON 형식으로 답하도록 고정
├─ auth.py	비밀번호 해시, 로그인 토큰, 아이디 규칙
├─ crypto.py	기록 암호화
├─ prompts/	AI 프롬프트
│ └─ principles.py	공통 원칙: 지침2에 따라 13개 항목 (관리자 페이지에서 항목별 수정 가능)
├─ base.py	전체 안내문 조합 + 고정 형식 요구사항
├─ intent_question.py	필요한 도움 확인 질문 (관리자 페이지에서 수정 가능)
├─ emotion_exploration.py	stabilization.py / recovery_plan.py / self_kindness.py
├─ router.py	네 가지 대화 흐름의 안내문(관리자 페이지에서 수정 가능)과 각 고정 형식 요구사항
├─ registry.py	사용자의 선택과 자기비판 정도에 따라 사용할 흐름 결정
├─ models/	데이터 테이블 정의
├─ schemas/	API 요청 / 응답 형식
├─ db/	DB 연결과 시작 시 초기화
└─ core/config.py	설정 항목 (.env 읽기)

API 그룹

경로	파일	내용
/api/session	session.py	대화 시작, 첫 질문, 대화 기록 목록, 언어 전환
/api/chat	chat.py	메시지 전송, AI 답변과 선택 카드 반환
/api/session/{id}/consent, /end	consent.py	사전 동의 항목 선택, 대화 종료 (보관에 동의하지 않으면 메시지 삭제)
/api/records	records.py	기록 저장, 본인의 전체 기록 조회, 삭제(본인 기록만)

경로	파일	내용
/api/resources	resources.py	위기 상황 도움 자원 (AI 생성이 아닌 고정 데이터)
/api/auth	auth.py	회원가입(가입 즉시 로그인), 로그인, 현재 사용자 조회
/api/admin	admin.py	관리자: 대화 데이터(필터, 내보내기), 사용자 관리(계정 만들기, 비밀번호 재설정, 필터, 사용 중지 / 복구, 역할, 삭제)
/api/admin/prompts	prompts.py	AI 대화 설정: 조회, 새 버전 저장, 수정 기록, 대화해 보기

주요 데이터 테이블

테이블	내용
users	아이디, 비밀번호 해시, 역할(일반 사용자 / 관리자 / 최고 관리자), 사용 여부
sessions	대화 단위: 소속 사용자, 언어, 사전 동의 항목, 사용자가 확인한 감정과 상황
messages	대화 메시지. AI 답변에는 당시 사용된 프롬프트 버전이 기록됨
saved_records	사용자가 저장한 기록 (암호화)
prompt_versions	관리자 페이지에서 수정한 프롬프트. 저장할 때마다 새 버전이 생기며 되돌릴 수 있음

현재 DB 마이그레이션 도구는 사용하지 않습니다. 시작 시 테이블을 자동 생성하고, 새 컬럼은 `db/session.py`에서 직접 추가합니다.

아이디는 `users` 테이블의 기존 `email` 컬럼에 저장됩니다. 따라서 예전에 이메일로 가입한 사용자는 그 이메일을 아이디로 계속 로그인할 수 있습니다. 예전의 `auth_tokens` 테이블은 더 이상 사용하지 않으며 그대로 두어도 됩니다.

3. 한 턴의 대화가 처리되는 과정

- 프론트엔드가 사용자의 입력을 `/api/chat`으로 보냅니다.
- 백엔드는 먼저 `safety.py`로 **키워드 검사**를 합니다. 위기 신호가 감지되면 AI를 거치지 않고 고정된 도움 안내를 바로 반환합니다.
- `prompts/router.py`가 사용자가 처음에 고른 항목(토로 / 감정 정리 / 안정화 / 대처 방법)과 자기비판 정도에 따라 **대화 흐름을 선택**합니다.
- AI에게 보낼 전체 안내문을 조합합니다: 13개 공통 원칙 + 흐름 안내문(이 두 부분은 관리자 페이지에서 수정 가능) + 고정 형식 요구사항(코드에 고정).
- `llm.py`가 모델을 호출하고, 「답변 문장 + 선택지」를 정해진 JSON 형식으로 받습니다.
- 결과를 프론트엔드에 전달해 채팅 말풍선과 선택 카드로 보여 줍니다. (현재는 사용자의 말만 검사하며, AI의 답변은 따로 검사하지 않습니다.)
- 사용자가 직접 고른 내용만 「확인된 정보」에 저장되며, AI의 추측은 바로 저장되지 않습니다.
- 요약, 계획, 나에게 건네는 말, 회복 행동을 확인한 뒤 사용자는 「내 기록에 저장」을 누를 수 있습니다. 저장을 누르면 해당 기록의 보관에 동의한 것으로 보며, 기록은 암호화되어 DB에 저장됩니다.

4. 관리자 페이지와 권한

기능	관리자	최고 관리자
대화 데이터 조회 (익명 코드, 아이디 미표시)	✓	✓
참여자·언어·날짜·메시지 수로 대화 필터	✓	✓
전체 / 필터 결과 / 참여자 한 명의 대화 내보내기 (사용자가 저장한 기록 포함)	✓	✓
각 대화에서 사용자가 저장한 기록 보기	✓	✓
사용자 목록	일반 사용자만 표시	일반 사용자와 관리자 표시
아이디·상태로 사용자 필터, 특정 사용자의 대화 내보내기	✓	✓ (역할로도 필터 가능)
계정 만들기	일반 사용자만	일반 사용자와 관리자
비밀번호 재설정, 사용 중지 / 복구, 사용자 삭제	일반 사용자만	일반 사용자와 관리자
관리자 지정 / 해제	×	✓
AI 대화 설정 수정	✓	✓

- 관리자 페이지에서는 누구도 최고 관리자 계정을 조작할 수 없습니다. 최고 관리자 지정은 DB를 직접 수정해야만 가능합니다.
- 사용자는 직접 회원가입(아이디 + 비밀번호, 가입 즉시 사용 가능)할 수도 있고, 관리자가 대신 계정을 만들어 줄 수도 있습니다.
- 이메일을 사용하지 않으므로 「비밀번호 찾기」 기능은 없습니다. 비밀번호를 잊은 사용자는 관리자에게 연락하고, 관리자가 재설정된 비밀번호를 전달받습니다.
- 내보내기 파일은 JSON 형식이며, 참여자는 익명 코드(예: **P-1a2b3c4d**)로 표시되고 아이디는 포함되지 않습니다.

5. 로컬 실행

```
# 백엔드
cd backend
cp .env.example .env          # OPENAI_API_KEY, JWT_SECRET,
                                RECORD_ENCRYPTION_KEY 등 입력
uv sync
uv run uvicorn app.main:app --reload --port 8000

# 프론트엔드
cd frontend
cp .env.example .env          # 백엔드 주소 지정
npm install
npm run dev                    # 기본 주소 http://localhost:5173
```